

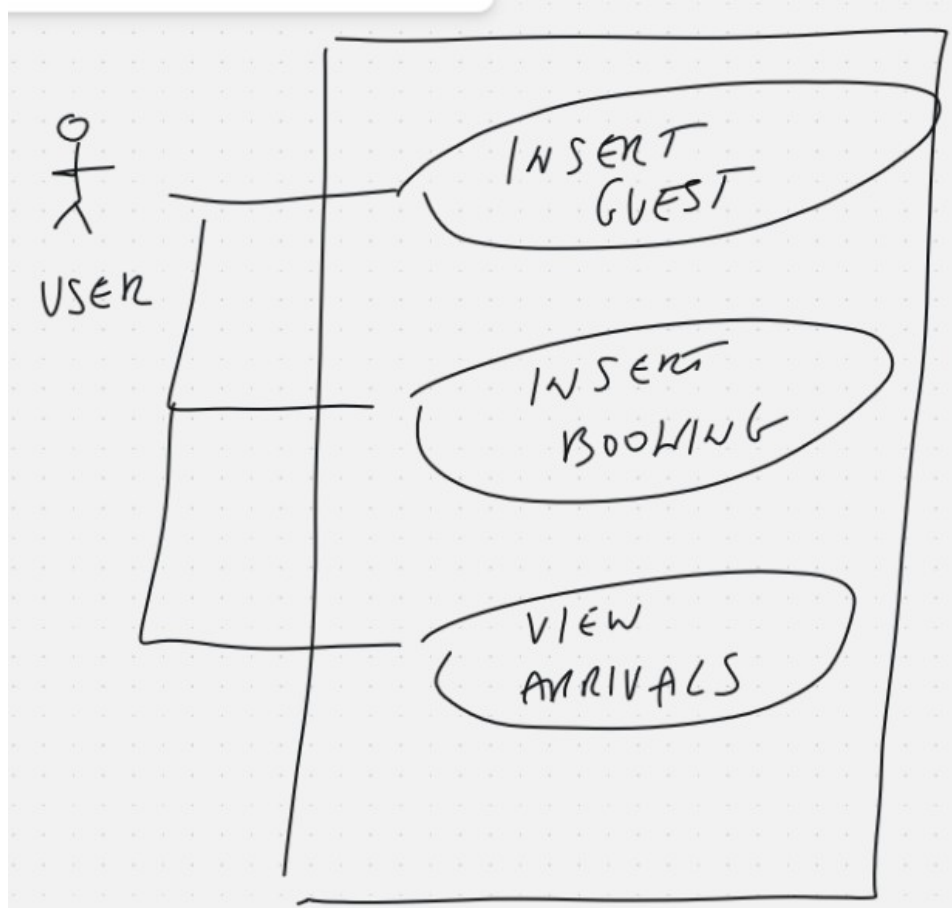
Appunti 25-02-26

Il problema delle dimensioni: le stanze sono tante.

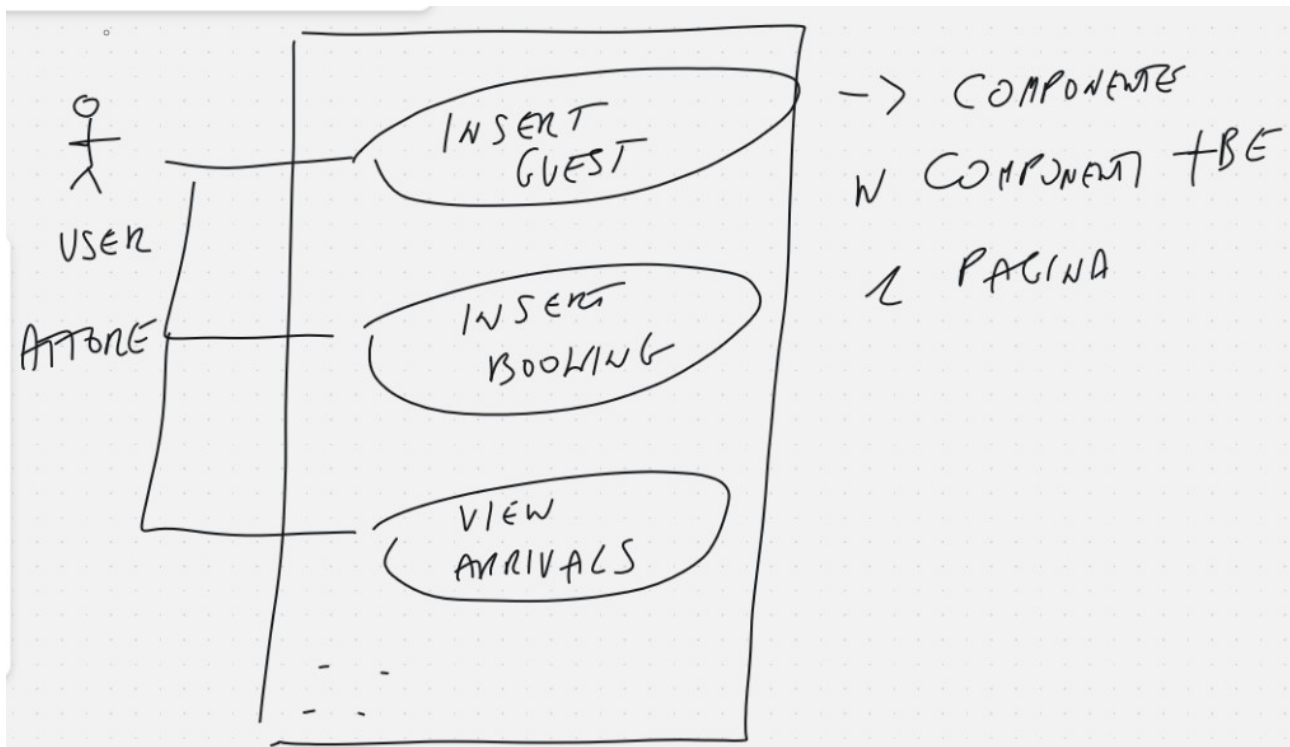
User: il receptionist di un certo hotel.

In base all'ospite che entra vedrò un hotel.

Gli USE CASE DIAGRAM vengono raggruppati per attore.



Attore = chiunque agisca sul sistema (essere umano / sistema informatico) .



La prima domanda che ci facciamo è :

(PROBLEM)
NECESSITA' : CHI ARRIVA OGGI NEL MIO HOTEL
(SOLUZIONE)

Che ho una api?

La prima cosa che facciamo è vedere se serve una api, e se serve prima la proviamo e poi la testiamo.

```
1 [
2   {
3     "from": "2026-02-25",
4     "guest": {
5       "address": "Via Kennedy, 122",
6       "city": "Genova",
7       "dob": "1992-12-01",
8       "firstName": "Luca",
9       "id": 1,
10      "lastName": "Brambilla",
11      "ssn": "VPXBMN17L74T768N"
12    },
13    "guestId": 1,
14    "id": 1,
15    "notes": "none",
16    "price": 90,
17    "room": {
18      "basePrice": 60.0,
19      "description": "Camera singola accogliente con Wi-Fi.",
20      "id": 1,
21      "name": "Camera 101"
22    },
23    "roomId": 1,
24    "to": "2026-02-26"
25  }
```

BOOKING DTO

Booking DTO manda due padri (room e guest)

```
GET http://localhost:8080/sbb/api/bookings/todayarrivals/1

Body
Status: 200 OK Time: 135 ms Size: 605 B

1 [
2   {
3     "from": "2026-02-25",
4     "guest": {
5       "address": "Via Kennedy, 122",
6       "city": "Genova",
7       "dob": "1992-12-01",
8       "firstName": "Luca",
9       "id": 1,
10      "lastName": "Brambilla",
11      "ssn": "VPXBMM17L74T760N"
12    },
13    "guestId": 1,
14    "id": 1,
15    "notes": "none",
16    "price": 90,
17    "room": {
18      "basePrice": 60.0,
19      "description": "Camera singola accogliente con Wi-Fi.",
20      "id": 1,
21      "name": "Camera 101"
22    },
23    "roomId": 1,
```

```
})
export class TodayArrivals {

  // BookingService
  private bookingService = inject(BookingService);

  // UserService
  // lo UserService carica utenti e mantiene un signal che contiene l'utente LOGGATO
  private userService = inject(UserService);

  loggedUser = this.userService.loggedUser;

  bookings = signal<Booking[]>([]);

  constructor(){
    effect(() => {
      const user = this.loggedUser();

      // Se l'utente c'è e ha un hotel ID, allora faccio la chiamata
      if (user && user.hotel?.id) {
        console.log("Caricamento per hotel:", user.hotel.id);

        this.bookingService.getTodayArrivals(user.hotel.id).subscribe({
          next: (json) => this.bookings.set(json),
          error: (err) => console.error(err)
        });
      }
    });
  }
};
```

Lo userService carica utenti e mantiene un signal che contiene l'utente loggato.

È stato simulato il login di Laura Leone, questa fase è di prova. Laura Leone è loggata, questo utente gestisce hotel numero 1.

Signal parametrizzato a User | Null → potrebbe esserci un utente loggato o potrebbe non esserci ancora.

```

constructor(){

    /*
        effect è un altro tipo di SIGNAL
        ed è una OPERAZIONE che viene eseguita quando un altro signal cambia
        computed ricalcola un valore
        effect esegue una operazione
        computed ritorna qualcosa
        effect lo puoi pensare come un metodo void
    */

    effect(() => {
        const user = this.loggedUser();

        // Se l'utente c'è e ha un hotel ID, allora faccio la chiamata
        if (user && user.hotel?.id) {
            console.log("Caricamento per hotel:", user.hotel.id);

            this.bookingService.getTodaysArrivals(user.hotel.id).subscribe({
                next: (json) => this.bookings.set(json),
                error: (err) => console.error(err)
            });
        }
    });
}

```

rc > app > booking-row > TS booking-row.ts > BookingRow

```

5    /*
6        Questo componente riceve un booking come input
7        e lo grafica
8        Questo in java sarebbe stato
9        viewBooking.render(booking)
10   */
11   @Component({
12       selector: 'app-booking-row',
13       imports: [CommonModule],
14       templateUrl: './booking-row.html',
15       styleUrls: ['./booking-row.css'],
16   })
17   export class BookingRow {
18
19       // ricevo un dato da graficare, come se fosse un parametro
20       // come fosse Antani
21
22       // input signal: un valore ricevuto da mio padre
23       // io sono un componente semplice
24       // vedo una prenotazione e la stampo
25       booking = input.required<Booking>();
26
27
28   }
29

```

```
6     }
7
8     <!-- Lista Arrivi -->
9     @for (booking of bookings(); track booking.id) {
10         <app-booking-row [booking]="booking"></app-booking-row>
11     }
12 </div>
13
```

RICHIESTA: realizzare partenze di oggi.